

Input Validation Cheat Sheet

Introduction

This article is focused on providing clear, simple, actionable guidance for providing Input Validation security functionality in your applications.

Goals of Input Validation

Input validation is performed to ensure only properly formed data is entering the workflow in an information system, preventing malformed data from persisting in the database and triggering malfunction of various downstream components. Input validation should happen as early as possible in the data flow, preferably as soon as the data is received from the external party.

Data from all potentially untrusted sources should be subject to input validation, including not only Internet-facing web clients but also backend feeds over extranets, from [suppliers](#), [partners](#), [vendors](#) or [regulators](#), each of which may be compromised on their own and start sending malformed data.

Input Validation should not be used as the *primary* method of preventing [XSS](#), [SQL Injection](#) and other attacks which are covered in respective [cheat sheets](#) but can significantly contribute to reducing their impact if implemented properly.

Input Validation Strategies

Input validation should be applied at both syntactic and semantic levels:

- **Syntactic** validation should enforce correct syntax of structured fields (e.g. SSN, date, currency symbol).
- **Semantic** validation should enforce correctness of their *values* in the specific business context (e.g. start date is before end date, price is within expected range).

It is always recommended to prevent attacks as early as possible in the processing of the user's (attacker's) request. Input validation can be used to detect unauthorized input before it is processed by the application.

Implementing Input Validation

Input validation can be implemented using any programming technique that allows effective enforcement of syntactic and semantic correctness, for example:

- Use data type validators available natively in web application frameworks (such as [Django Validators](#), [Apache Commons Validators](#) etc).
- Validation against [JSON Schema](#) and [XML Schema \(XSD\)](#) for input in these formats.
- Type conversion (e.g. `Integer.parseInt()` in Java, `int()` in Python) with strict exception handling
- Minimum and maximum value range check for numerical parameters and dates, minimum and maximum length check for strings.
- Array of allowed values for small sets of string parameters (e.g. days of week).
- Regular expressions for any other structured data covering the whole input string (`^...$`) and **not** using "any character" wildcard (such as `.` or `\S`)
- Denylisting known dangerous patterns can be used as an additional layer of defense, but it should supplement - not replace - allowlisting, to help catch some commonly observed attacks or patterns without relying on it as the main validation method.

Allowlist vs Denylist

It is a common mistake to use denylist validation in order to try to detect possibly dangerous characters and patterns like the apostrophe `'` character, the string `1=1`, or the `<script>` tag, but this is a massively flawed approach as it is trivial for an attacker to bypass such filters.

Plus, such filters frequently prevent authorized input, like `O'Brian`, where the `'` character is fully legitimate. For more information on XSS filter evasion please see [this wiki page](#).

While denylisting can be useful as an additional layer of defense to catch some common malicious patterns, it should not be relied upon as the primary method. Allowlisting remains the more robust and secure approach for preventing potentially harmful input.

Allowlist validation is appropriate for all input fields provided by the user. Allowlist validation involves defining exactly what IS authorized, and by definition, everything else is not authorized.

If it's well structured data, like dates, social security numbers, zip codes, email addresses, etc. then the developer should be able to define a very strong validation pattern, usually based on regular expressions, for validating such input.

If the input field comes from a fixed set of options, like a drop down list or radio buttons, then the input needs to match exactly one of the values offered to the user in the first place. Any failure to validate a value against this discrete list of options on the server side is a high security event and should be logged as a high severity event as it indicates that an attacker is tampering with the client-side code.

Validating Free-form Unicode Text

Free-form text, especially with Unicode characters, is perceived as difficult to validate due to a relatively large space of characters that need to be allowed.

It's also free-form text input that highlights the importance of proper context-aware output encoding and quite clearly demonstrates that input validation is **not** the primary safeguards against Cross-Site Scripting. If your users want to type apostrophe ' or less-than sign < in their comment field, they might have perfectly legitimate reason for that and the application's job is to properly handle it throughout the whole life cycle of the data.

The primary means of input validation for free-form text input should be:

- **Normalization:** Ensure canonical encoding is used across all the text and no invalid characters are present.
- **Character category allowlisting:** Unicode allows listing categories such as "decimal digits" or "letters" which not only covers the Latin alphabet but also various other scripts used globally (e.g. Arabic, Cyrillic, CJK ideographs etc).
- **Individual character allowlisting:** If you allow letters and ideographs in names and also want to allow apostrophe ' for Irish names, but don't want to allow the whole punctuation category.

References:

- [Input validation of free-form Unicode text in Python](#)
- [UAX 31: Unicode Identifier and Pattern Syntax](#)
- [UAX 15: Unicode Normalization Forms](#)
- [UAX 24: Unicode Script Property](#)

Regular Expressions (Regex)

Developing regular expressions can be complicated, and is well beyond the scope of this cheat sheet.

There are lots of resources on the internet about how to write regular expressions, including this [site](#) and the [OWASP Validation Regex Repository](#).

When designing regular expressions, be aware of [RegEx Denial of Service \(ReDoS\) attacks](#). These attacks cause a program using a poorly designed Regular Expression to operate very slowly and utilize CPU resources for a very long time.

In summary, input validation should:

- Be applied to all input data, at minimum.

- Define the allowed set of characters to be accepted.
- Define a minimum and maximum length for the data (e.g. `{1,25}`).

Allowlist Regular Expression Examples

Validating a U.S. Zip Code (5 digits plus optional -4)

```
^\d{5}(-\d{4})?$
```

Validating U.S. State Selection From a Drop-Down Menu

```
^(AA|AE|AP|AL|AK|AS|AZ|AR|CA|CO|CT|DE|DC|FM|FL|GA|GU|
HI|ID|IL|IN|IA|KS|KY|LA|ME|MH|MD|MA|MI|MN|MS|MO|MT|NE|
NV|NH|NJ|NM|NY|NC|ND|MP|OH|OK|OR|PW|PA|PR|RI|SC|SD|TN|
TX|UT|VT|VI|VA|WA|WV|WI|WY)$
```

Java Regex Usage Example:

Example validating the parameter "zip" using a regular expression.

```
private static final Pattern zipPattern = Pattern.compile("^\d{5}(-\d{4})?$");

public void doPost( HttpServletRequest request, HttpServletResponse response)
{
    try {
        String zipCode = request.getParameter( "zip" );
        if ( !zipPattern.matcher( zipCode ).matches() ) {
            throw new YourValidationException( "Improper zipcode format." );
        }
        // do what you want here, after its been validated ..
    } catch(YourValidationException e ) {
        response.sendError( response.SC_BAD_REQUEST, e.getMessage() );
    }
}
```

Some Allowlist validators have also been predefined in various open source packages that you can leverage. For example:

- [Apache Commons Validator](#)

Client-side vs Server-side Validation

Input validation **must** be implemented on the server-side before any data is processed by an application's functions, as any JavaScript-based input validation performed on the client-side can be circumvented by an attacker who disables JavaScript or uses a web proxy.

Implementing both client-side JavaScript-based validation for UX and server-side validation for security is the recommended approach, leveraging each for their respective strengths.

Validating Rich User Content

It is very difficult to validate rich content submitted by a user. For more information, please see the XSS cheat sheet on [Sanitizing HTML Markup with a Library Designed for the Job](#).

Preventing XSS and Content Security Policy

All user data controlled must be encoded when returned in the HTML page to prevent the execution of malicious data (e.g. XSS). For example `<script>` would be returned as

```
&lt;script&gt;
```

The type of encoding is specific to the context of the page where the user controlled data is inserted. For example, HTML entity encoding is appropriate for data placed into the HTML body. However, user data placed into a script would need JavaScript specific output encoding.

Detailed information on XSS prevention here: [OWASP XSS Prevention Cheat Sheet](#)

File Upload Validation

Many websites allow users to upload files, such as a profile picture or more. This section helps provide that feature securely.

Check the [File Upload Cheat Sheet](#).

Upload Verification

- Use input validation to ensure the uploaded filename uses an expected extension type.
- Ensure the uploaded file is not larger than a defined maximum file size.
- If the website supports ZIP file upload, do a validation check before unzipping the file. The check includes the target path, level of compression, estimated unzip size.

Upload Storage

- Use a new filename to store the file on the OS. Do not use any user controlled text for this filename or for the temporary filename.
- When the file is uploaded to web, it's suggested to rename the file on storage. For example, the uploaded filename is `test.JPG`, rename it to `JA1287uaisdjh.JPG` with a random

filename. The purpose of doing it to prevent the risks of direct file access and ambiguous filename to evade the filter, such as `test.jpg;.asp` or `../../../../../../../../test.jpg`.

- Uploaded files should be analyzed for malicious content (anti-malware, static analysis, etc).
- The client should not be able to specify the file path; it should be defined by the server.

Public Serving of Uploaded Content

- Ensure uploaded images are served with the correct content-type (e.g. `image/jpeg`, `image/png`)

Beware of Specific File Types

The upload feature should be using an allowlist approach to only allow specific file types and extensions. However, it is important to be aware of the following file types that, if allowed, could result in security vulnerabilities:

- **crossdomain.xml** / **clientaccesspolicy.xml**: allows cross-domain data loading in Flash, Java and Silverlight. If permitted on sites with authentication this can permit cross-domain data theft and CSRF attacks. Note this can get pretty complicated depending on the specific plugin version in question, so it's best to just prohibit files named "crossdomain.xml" or "clientaccesspolicy.xml".
- **.htaccess** and **.htpasswd**: Provides server configuration options on a per-directory basis, and should not be permitted. See [HTACCESS documentation](#).
- Web executable script files are suggested not to be allowed such as `aspx`, `asp`, `css`, `swf`, `xhtml`, `rhtml`, `shtml`, `jsp`, `js`, `pl`, `php`, `cgi`.

Image Upload Verification

- Use image rewriting libraries to verify the image is valid and to strip away extraneous content.
- Set the extension of the stored image to be a valid image extension based on the detected content type of the image from image processing (e.g. do not just trust the header from the upload).
- Ensure the detected content type of the image is within a list of defined image types (jpg, PNG, etc)

Email Address Validation

Syntactic Validation

The format of email addresses is defined by [RFC 5321](#), and is far more complicated than most people realise. As an example, the following are all considered to be valid email addresses:

- `"<script>alert(1);</script>"@example.org`
- `user+subaddress@example.org`
- `user@[IPv6:2001:db8::1]`
- `" "@example.org`

Properly parsing email addresses for validity with regular expressions is very complicated, although there are a number of [publicly available documents on regex](#).

The biggest caveat on this is that although the RFC defines a very flexible format for email addresses, most real world implementations (such as mail servers) use a far more restricted address format, meaning that they will reject addresses that are *technically* valid. Although they may be technically correct, these addresses are of little use if your application will not be able to actually send emails to them.

As such, the best way to validate email addresses is to perform some basic initial validation, and then pass the address to the mail server and catch the exception if it rejects it. This means that the application can be confident that its mail server can send emails to any addresses it accepts. The initial validation could be as simple as:

- The email address contains two parts, separated with an @ symbol.
- The email address does not contain dangerous characters (such as backticks, single or double quotes, or null bytes).
 - Exactly which characters are dangerous will depend on how the address is going to be used (echoed in page, inserted into database, etc).
- The domain part contains only letters, numbers, hyphens (-) and periods (.).
- The email address is a reasonable length:
 - The local part (before the @) should be no more than 63 characters.
 - The total length should be no more than 254 characters.

Semantic Validation

Semantic validation is about determining whether the email address is correct and legitimate. The most common way to do this is to send an email to the user, and require that they click a link in the email, or enter a code that has been sent to them. This provides a basic level of assurance that:

- The email address is correct.
- The application can successfully send emails to it.

- The user has access to the mailbox.

The links that are sent to users to prove ownership should contain a token that is:

- At least 32 characters long.
- Generated using a [secure source of randomness](#).
- Single use.
- Time limited (e.g, expiring after eight hours).

After validating the ownership of the email address, the user should then be required to authenticate on the application through the usual mechanism.

Disposable Email Addresses

In some cases, users may not want to give their real email address when registering on the application, and will instead provide a disposable email address. These are publicly available addresses that do not require the user to authenticate, and are typically used to reduce the amount of spam received by users' primary email addresses.

Blocking disposable email addresses is almost impossible, as there are a large number of websites offering these services, with new domains being created every day. There are a number of publicly available lists and commercial lists of known disposable domains, but these will always be incomplete.

If these lists are used to block the use of disposable email addresses then the user should be presented with a message explaining why they are blocked (although they are likely to simply search for another disposable provider rather than giving their legitimate address).

If it is essential that disposable email addresses are blocked, then registrations should only be allowed from specifically-allowed email providers. However, if this includes public providers such as Google or Yahoo, users can simply register their own disposable address with them.

Sub-Addressing

Sub-addressing allows a user to specify a *tag* in the local part of the email address (before the @ sign), which will be ignored by the mail server. For example, if that `example.org` domain supports sub-addressing, then the following email addresses are equivalent:

- `user@example.org`
- `user+site1@example.org`
- `user+site2@example.org`

Many mail providers (such as Microsoft Exchange) do not support sub-addressing. The most notable provider who does is Gmail, although there are many others that also do.

Some users will use a different *tag* for each website they register on, so that if they start receiving spam to one of the sub-addresses they can identify which website leaked or sold their email address.

Because it could allow users to register multiple accounts with a single email address, some sites may wish to block sub-addressing by stripping out everything between the `+` and `@` signs. This is not generally recommended, as it suggests that the website owner is either unaware of sub-addressing or wishes to prevent users from identifying them when they leak or sell email addresses. Additionally, it can be trivially bypassed by using [disposable email addresses](#), or simply registering multiple email accounts with a trusted provider.

References

- [OWASP Top 10 Proactive Controls 2024: C3: Validate all Input & Handle Exceptions](#)
- [CWE-20 Improper Input Validation](#)
- [OWASP Top 10 2021: A03:2021-Injection](#)
- [Snyk: Improper Input Validation](#)